

Algorithmic Trading on Energy Spreads: A Data-Driven Implementation in Python

Personal Project - Quantitative Development

February 2026

1 Market Fundamentals: The WTI/Brent Spread

The spread between West Texas Intermediate (WTI) and Brent Blend is the primary benchmark for pricing global crude oil. Although both are "Light Sweet" crudes, they are driven by distinct geographical and logistical factors that justify a statistical arbitrage approach.

1.1 Benchmark Characteristics

- **WTI (NYMEX):** Sourced primarily from the Permian Basin and delivered at the Cushing, Oklahoma hub. As a landlocked benchmark, its pricing is highly sensitive to US domestic pipeline capacity and storage levels at Cushing.
- **Brent (ICE):** A seaborne grade from the North Sea, serving as the reference for approximately two-thirds of internationally traded crude. It reflects global geopolitical risks and maritime freight costs.

1.2 Economic Drivers and Spread Dynamics

The spread S_t at time t is defined as the price difference between the two front-month futures contracts:

$$S_t = P_{WTI,t} - P_{Brent,t}$$

Historically, WTI traded at a premium due to its higher API gravity. However, following the US shale revolution and the 2015 lifting of the export ban, WTI has structurally moved to a discount relative to Brent. The spread fluctuates based on:

1. **Logistical Constraints:** Bottlenecks in US infrastructure often expand the WTI discount.
2. **Geopolitical Risk:** Brent is more reactive to Middle Eastern tensions and OPEC+ decisions.
3. **Shipping Costs:** As Brent is seaborne, tanker rates directly impact its relative value.

1.3 Statistical Arbitrage Thesis

The strategy relies on the principle of **co-integration**. Since these two grades are global substitutes for refiners, their prices cannot diverge indefinitely without creating an economic imbalance. Our model identifies transient "dislocations" where the spread deviates from its fundamental mean, betting on a return to equilibrium.

Executive Summary

This project focuses on the implementation of a systematic trading bot on the WTI/Brent spread. Moving away from theoretical assumptions, the strategy relies on a robust Data Science approach: rigorous data cleaning (handling asynchronous market hours), vectorised backtesting with Pandas, and strict Out-of-Sample validation. The algorithm achieved a Sharpe Ratio of 2.17 on unseen data (2024-2026).

2 Theoretical Framework: Mean Reversion

The strategy is based on the **Mean Reversion** principle, a financial theory suggesting that asset prices (or spreads) eventually return to their long-term average. In the context of the WTI/Brent spread, we assume that while prices can diverge due to temporary shocks, the fundamental co-integration of these benchmarks forces the spread back to its mean.

2.1 Standardized Deviation (Z-Score)

To quantify the deviation of the current spread S_t from its historical average, we calculate a dynamic Z-Score:

$$Z_t = \frac{S_t - \text{MA}_n(S_t)}{\sigma_n(S_t)}$$

Where:

- MA_n is the n -period moving average of the spread.
- σ_n is the n -period rolling standard deviation.

2.2 Trading Signal Logic

The core assumption is that a Z-Score exceeding a specific threshold indicates a **statistical anomaly**. The model operates on the following probabilities:

- **Expansion** ($|Z_t| > 1.9$): The spread is considered "overextended". We take a position betting on the contraction towards the mean.
- **Convergence** ($|Z_t| \rightarrow 0$): As the spread returns to its average, the position is liquidated to capture the profit from the normalization.

This approach transforms a simple price difference into a stationary signal, allowing for systematic entry and exit points.

3 Data Engineering & Cleaning

The reliability of a backtest depends on the quality of the data. Retrieving data from `yfinance` required specific processing to align the NYMEX (WTI) and ICE (Brent) calendars.

- **Synchronization:** I implemented an inner-join logic ('dropna') to remove days where one market was closed (e.g., US Labor Day vs UK Bank Holidays), preventing the algorithm from trading on "stale" prices.
- **Feature Engineering:** The Z-Score was calculated on a rolling window to normalize the spread volatility dynamically.

4 Core Algorithm Implementation

The strategy is implemented as a vectorised state machine to ensure execution speed and avoid "look-ahead bias". Below is the core logic used to manage positions, specifically handling the "Hold" state via forward-filling methods.

```

1 # Step A: Initialize signal container with NaN
2 data['position'] = np.nan
3
4 # Step B: Entry Signals (Z-Score Thresholds)
5 # Long if Spread is statistically undervalued
6 data.loc[data['z_score'] < -1.9, 'position'] = 1
7 # Short if Spread is statistically overvalued

```

```

8 data.loc[data['z_score'] > 1.9, 'position'] = -1
9
10 # Step C: Exit Conditions (Mean Reversion)
11 # We exit when the price returns to equilibrium (|Z| < 0.5)
12 data.loc[data['z_score'].abs() < 0.5, 'position'] = 0
13
14 # Step D: Risk Management (Stop Loss)
15 # Hard exit if volatility explodes (Z > 3.5)
16 data.loc[data['z_score'].abs() > 3.5, 'position'] = 0
17
18 # Step E: State Propagation (The Logic Fix)
19 # We propagate the last valid signal forward to maintain positions
20 data['position'] = data['position'].ffill()
21
22 # Step F: Initial State
23 data['position'] = data['position'].fillna(0)

```

Listing 1: Python Implementation of the Signal Logic

This specific implementation using `ffill()` ensures that the bot stays in position until a clear exit signal (0) is triggered, solving the common issue of "flickering" signals in backtests.

5 Optimization & Robustness

To ensure the model generalizes well to new data:

1. **Avoiding Overfitting:** I ran a Grid Search on 2000-2023 data but deliberately selected the **third-best parameter set** rather than the absolute maximum. This parameter set showed better stability across nearby values.
2. **Out-of-Sample Test:** The model was then run on strictly unseen data (Jan 2024 - Present).

6 Performance Results (Out-of-Sample)

- **Sharpe Ratio:** 2.17
- **Net PnL:** \$14.60 per barrel (fees included)

6.1 Performance Evaluation: The Sharpe Ratio

To evaluate the efficiency of the strategy, we utilize the **Sharpe Ratio**, which measures the risk-adjusted return. It quantifies how much excess return the strategy generates for each unit of volatility (risk). The ratio is defined as:

$$S = \frac{E[R_p - R_f]}{\sigma_p}$$

Where:

- R_p is the strategy return.
- R_f is the risk-free rate (assumed to be zero for this intraday/short-term spread analysis).
- σ_p is the standard deviation (volatility) of the strategy's returns.

A Sharpe Ratio above 1.0 is generally considered good, while a ratio above 2.0 (as achieved in our out-of-sample backtest) indicates superior risk-adjusted performance, suggesting that the returns are consistent and not driven by isolated volatile events.

RESULTS (2024 - Today)
 Net PnL per Barrel : 14.60 \$
 Sharpe Ratio : 2.17

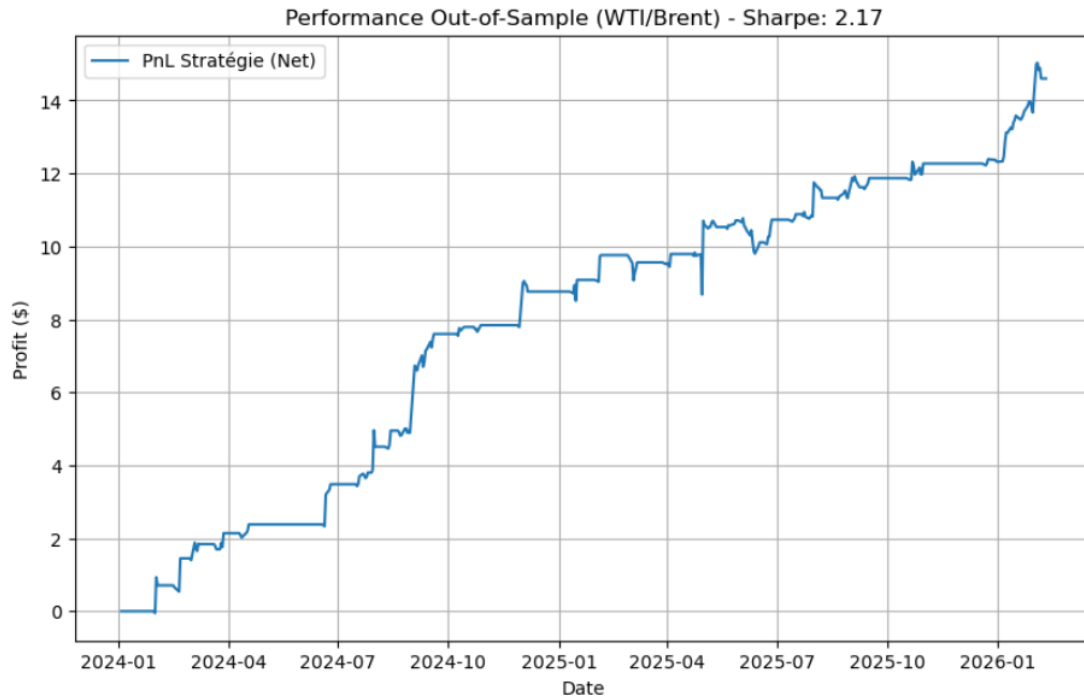


Figure 1: Equity Curve generated by Matplotlib on the test set.

7 Critical Analysis & Real-World Constraints

While the strategy performs well Out-of-Sample, a rigorous quantitative approach requires acknowledging inherent limitations suitable for live deployment:

- **Liquidity & Slippage:** The model assumes a fixed cost of \$0.05/bbl. In a live environment, entering positions during high volatility (e.g., OPEC announcements) could incur higher slippage. Future improvements would involve optimizing execution timing (TWAP/VWAP).
- **Macro Factors:** The model is purely statistical. Integrating exogenous data (inventory levels) could reduce false positives during structural market shifts.
- **End-of-Day Execution Assumption:** The current backtest relies exclusively on daily closing prices (EOD data). Assuming that trades can be executed exactly at the daily close is a theoretical simplification that abstracts away intraday volatility and liquidity constraints at the market close. Although a \$0.05 transaction cost is included to simulate slippage, real-world execution would require intraday tick data and specific routing algorithms (such as TWAP or VWAP) to minimize market impact.

Conclusion: A First Step into Commodity Trading

This project marks my first concrete step into the world of Commodity Trading. It allowed me to move from theoretical concepts to the operational reality of energy markets, specifically the WTI/Brent spread dynamics.

Through this implementation, I confronted two major industry challenges:

- **Data Reality:** Managing the asynchronous calendars of NYMEX and ICE was a practical lesson in the importance of clean data infrastructure before any modeling can occur.

- **Robustness over Hype:** By prioritizing parameter stability (3rd best) over raw historical performance, I adopted a risk-aware approach essential for long-term survival in the markets.

This experience confirms my strong interest in market dynamics and my readiness to deepen these skills within a professional trading desk.

Technology Stack: Python (Pandas, NumPy, Matplotlib), yfinance API.

Code Repository: github.com/adebove/WTI-BRENT-Spread-Trading